

Lab 3: Component Modelling & Architectural Pattern Selection

System: OpenAPI Mock Server Generator

Architecture Selection

We chose Layered Architecture for the OpenAPI Mock Server Generator System.

The system is organised into three horizontal layers. The Presentation Layer (Web Console UI and Mock API Gateway) is the only entry point for API Developers, QA Engineers and the CI/CD pipeline. The Business Layer holds the Spec Parser & Validator, Endpoint Generator, Request Validator, Response Generator and Logging Service. The Data Layer holds the Spec & Mock Definition Repository and the Request Log Store. Each layer only calls the layer directly below it, through the provided and required interfaces shown in the component diagram.

Comparison of Styles Considered

Architectural Style	Fit for the OpenAPI Mock Server Generator	Decision
Layered	Processing is a pipeline (spec → parse → generate → validate → respond); every stage maps to one component behind one interface.	Selected
Microservices	Independent scaling is not needed; network hops would add latency to mock responses; needs service discovery and per-service deployment.	Rejected
Client-Server	Describes how clients reach the tool but gives no structure for the internal modules; the server would become one unstructured block.	Rejected as main style

Two Reasons for the Choice

1. **The workflow is naturally top-down.** Use cases UC01–UC07 follow one path: upload the spec, parse it, generate endpoints, then validate each request and return a schema-compliant response. Every step becomes one component with a clear interface (IUploadSpec, IParsedSpec, IValidateRequest, IGenerateResponse), so a component such as the parser can be replaced, for example to support a newer OpenAPI version, without changing the Gateway or the UI.
2. **It is right-sized for a developer and QA tool.** The generator runs as one deployable unit that a team can start locally or inside a CI/CD job. Layering keeps the code easy to understand, build and test layer by layer, without the service discovery, container orchestration and distributed debugging that microservices would add to a small tool.

Security Advantage

All external input enters through the Presentation Layer and must pass through the Business Layer before reaching storage. Uploaded specifications are checked by the Spec Parser & Validator (file size limit, schema check, no unsafe YAML tags, no remote \$ref loading), and every mock call is checked by the Request Validator before any response is built, so malformed or malicious input is rejected early. The Data Layer is never exposed to clients directly, and authentication and rate limiting can be applied at a single point, the Gateway.

Performance Benefit

Layers communicate through in-process calls, so validation and response generation add no network hops. The parsed specification and route definitions are cached in memory after the first load, and the Request Validator and Response Generator only perform reads. As a result, the response time seen by a QA Engineer is dominated by the latency configured in UC06 rather than by the architecture, which keeps timeout and performance tests predictable.